

PCS956-TS-02-forecasting-and-validation

September 24, 2026

1 PCS956 - Time Series for ML - Forecasting Baselines, Models, Temporal Validation, and Spectral Views

Note on code cells: Each code cell in this lecture is intended to be self-contained: it defines any variables it uses. You can run cells independently (or copy them into your own notebooks) without relying on hidden state from earlier cells.

This lecture builds on TS1. We assume you are familiar with:

- basic time series concepts (temporal indexing, trend, seasonality, residuals);
- visual EDA and decomposition (TS1 Sections 4-5);
- train/validation/test splits, leakage, and quarantine gaps (TS1 Section 4.4);
- stationarity and random-walk intuition (TS1 Section 6).

Note on mathematics and further reading In this module we emphasise concepts and practical workflows rather than detailed mathematical theory. We will use some terminology from probability and statistics (for example ‘stationarity’, ‘autocorrelation’, ‘white noise’), but we do not assume a strong background in stochastic processes.

Where we refer to more formal results, we will usually state them informally and focus on how they influence modelling and validation choices. Students who would like to see more formal definitions and worked examples, or explore some additional methods, are encouraged to consult:

- Rob J. Hyndman and George Athanasopoulos, *Forecasting: Principles and Practice* (3rd ed.), available online at <https://otexts.com/fpp3/>.

That online book provides:

- a careful introduction to time series concepts;
- more detailed treatments of ARIMA models, seasonality, and spectral methods;
- practical forecasting examples with code.

Here we focus on:

- practical forecasting baselines (naive, persistence, seasonal, simple AR);
- classical models (ARMA/ARIMA) used as baselines rather than magical solutions;
- casting time series forecasting as supervised learning with lagged features;
- applying ML models (e.g. tree-based) to lagged features with **time-aware validation**;
- evaluating models on levels vs differences relative to strong baselines;
- a short spectral view (periodograms) for seasonality and structure.

The companion notebook PCS956-TS-companion_B contains **code templates** for baselines (mean, persistence, seasonal naive, simple AR), ARIMA, ML on lagged features, time-aware

train/validation/test splits, evaluation on levels vs differences, and simple spectral views via periodograms. It is intended as a starting point for the mini-project.

1.1 1. Recap and motivation

Lecture TS1 introduced: - basic time series concepts: temporal index, finite samples, univariate vs multivariate series; - data quality, anomalies, rare events, and regime changes; - visual EDA: plotting levels and differences, trend and seasonality, autocorrelation; - decomposition into deterministic (trend, seasonality) and stochastic components; - stationarity, random-walk intuition, and modelling pitfalls.

A central message was scepticism: not all series contain exploitable structure, and even when they do, naive modelling can produce illusions of skill.

1.1.1 1.1 From inspection to responsible forecasting

Time series forecasting is one of the main motivations for investigating time series, and many algorithms have been developed for this purpose. However, forecasting methods can easily be misused if we ignore:

- the temporal dependence structure;
- non-stationarity and regime changes;
- the need for appropriate transformations and validation schemes.

Before moving to forecasting pipelines, we must ask:

- does the series exhibit any exploitable structure at all;
- is there trend, seasonality, or dependence beyond simple persistence;
- do residuals after decomposition appear noise-like, or do they clearly contain remaining structure.

Residuals that look ‘white-noise-like’ under simple visual inspection (time plots, autocorrelation functions, or basic spectral density estimates) can still hide non-linear or higher-order structure (for example changing variance or conditional dependence). Even simple transformations, such as plotting squared residuals to look for structure in volatility, may reveal patterns that are not visible on the original scale, but these checks are heuristic rather than definitive tests.

When we talk about forecasting we must also be clear about the forecast horizon: how far ahead we want to predict. One-step-ahead and multi-step forecasts can have very different properties, and the same model may perform well for short horizons but poorly when we look several steps ahead. The objective that we optimise (for example one-step mean squared error versus multi-step error over a longer window) and the computational cost of producing forecasts (for instance, whether a model can deliver predictions within the time available in an operational setting) will both influence model choice and how we interpret performance.

This lecture builds on the basic inspection and decomposition from Lecture TS1 and focuses on how to move from ‘cleaned and inspected series’ to responsible forecasting pipelines that respect time order, use strong baselines, and avoid naive implementations.

1.2 2. Baselines for forecasting

Baselines are central tools for forecasting, not mere toy examples. They answer the question: ‘How much can we achieve with simple structure, without complex ML models?’

Before applying complex models, we should ask whether there is any structure to exploit beyond simple baselines:

- white noise:
 - in the classical sense, observations are i.i.d.;
 - apart from trivial forecasting of the mean (or distribution), there is no predictable structure in the sequence itself;
 - in practice, data that look noise-like in simple plots, autocorrelation functions, or basic spectral estimates may still hide non-linear or higher-order dependence (for example conditional heteroskedasticity).
- random walk:
 - $y_t = y_{t-1} + e_t$ with shocks e_t ;
 - strong autocorrelation in levels, variance grows over time;
 - one-step-ahead persistence forecasts $\hat{y}_{t+1} = y_t$ are optimal for a pure random walk.

Simple standard methods can often ‘do the trick’ when: - the series behaves approximately like a random walk; - the main structure is trend and seasonality with weak additional dependence.

In these cases, naive or seasonal baselines and simple AR models form powerful benchmarks. More complex models must be compared against them on the same targets and evaluation windows, and any claimed improvement should be checked carefully to see whether it reflects genuinely better forecasts or simply overfitting or small, unstable gains when the residuals or errors already appear close to noise under basic diagnostics.

1.2.1 2.1 Naive and persistence forecasts

Naive and persistence forecasts are simple but surprisingly strong baselines:

- mean baseline:
 - always predict the sample mean over a training period;
 - useful for series that look roughly white-noise-like, with no obvious trend or dependence.
- naive/persistence baseline:
 - for one-step-ahead forecasting, set $\hat{y}_{t+1} = y_t$;
 - for multi-step horizons, extend this idea (for example repeat the last observed value).

For pure random walks, one-step-ahead persistence forecasts are optimal in the mean-square-error sense. Even in more realistic settings, persistence often performs well. Any proposed model should be compared against this baseline; failure to beat persistence indicates limited exploitable structure or poor model design. For longer forecast horizons, persistence still provides a natural reference, but different models may dominate at different horizons, and comparisons should be made at the specific horizons that matter for the application.

1.2.2 2.2 Seasonal naive forecasts

Many series exhibit strong seasonality (daily, weekly, yearly). Seasonal naive forecasts extend persistence by repeating the last observed value from the same point in the seasonal cycle:

- daily seasonality:
 - forecast tomorrow’s value by using the value from the same time yesterday;
- weekly seasonality:
 - forecast next Monday’s value by using last Monday’s value.

Formally, for a seasonal period m ,

$$\hat{y}_{t+1} = y_{t+1-m},$$

where m might be 7 for weekly seasonality in daily data, or 24 for hourly data with daily cycles.

Seasonal naive forecasts can be extremely competitive when seasonality dominates other structures. Any more complex seasonal model must demonstrate improvement over these baselines.

1.2.3 2.3 Simple autoregressive baselines

Simple autoregressive (AR) models provide next-step benchmarks that capture linear dependence over a small number of lags. An $AR(p)$ model has the form:

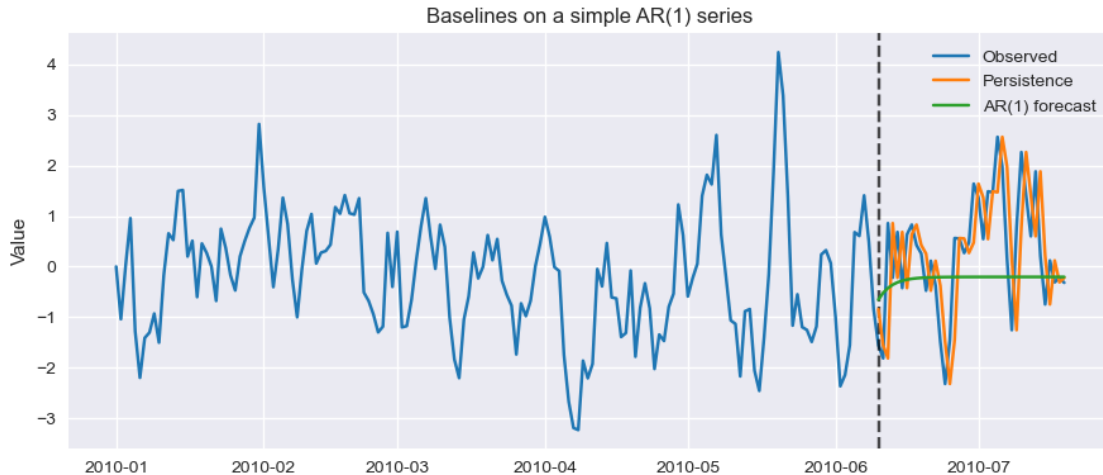
$$y_t = \phi_1 y_{t-1} + \phi_2 y_{t-2} + \dots + \phi_p y_{t-p} + e_t,$$

where e_t is a noise term and $\phi_1, \dots, \phi_p$ are coefficients.

Conceptually: - AR models are linear models with memory; - they can be viewed as regressions on lagged values; - they bridge classical time series modelling and generic supervised learners on lagged features.

Simple AR baselines (for example $AR(1)$ or $AR(2)$) often capture much of the predictable linear structure. More flexible models should deliver clear benefits beyond these baselines, especially when evaluated with proper temporal validation.

	MAE	RMSE
Mean baseline	0.987184	1.217204
Persistence	0.860139	1.050492
AR(1)	0.982390	1.209258



1.3 3. Stationarity, random walks, and differencing

This section revisits random-walk behaviour and unit roots, and introduces differencing as a key tool. The goal is to understand: - when persistence baselines are essentially optimal; - when differencing reveals that little structure remains beyond white noise; - how levels vs differences affect our perception of forecasting skill.

1.3.1 3.1 Random-walk behaviour and unit roots

A simple random walk can be written as:

$$y_t = y_{t-1} + e_t,$$

where e_t are shocks, often taken as independent with mean zero. Properties:

- strong autocorrelation in levels: y_t is close to y_{t-1} ;
- variance grows with time;
- non-stationary in levels, but differences $d_t = y_t - y_{t-1}$ may be stationary.

In unit-root terminology, such processes have a characteristic root on the unit circle. For pure random walks, one-step-ahead persistence forecasts $\hat{y}_{t+1} = y_t$ are optimal. This creates:

- illusions of skill on levels: simply copying yesterday can look excellent;
- difficulty for more complex models to beat persistence, especially in short samples.

Recognising random-walk-like behaviour is therefore central to realistic expectations about forecasting performance.

1.3.2 3.2 Differencing as a tool

Differencing transforms levels into changes:

$$d_t = y_t - y_{t-1}.$$

For random walks,

$$d_t = (y_{t-1} + e_t) - y_{t-1} = e_t,$$

so a single difference exactly recovers the noise sequence in this idealised case. More generally:

- differencing is used to remove low-frequency structure such as trends;
- the aim is to obtain a series that is approximately stationary, not necessarily pure white noise;
- some series require higher-order differencing (for example taking second differences) before the transformed series is reasonably stationary.

A common strategy is:

- decompose the series into trend, seasonality, and residuals;

- apply transformations if needed (for example differencing, log transforms);
- inspect residuals for remaining structure.

Key questions: - do residuals (or differences) look approximately white-noise-like under basic diagnostics, suggesting limited remaining predictive structure; - or do residuals show clear autocorrelation or other patterns, indicating that further modelling may be useful.

It is important to remember that ‘looking like white noise’ in plots or simple summaries is not the same as truly being white noise. Finite samples can hide lingering structure, and more subtle forms of dependence may remain even when autocorrelation appears weak. Conversely, forcing increasingly complex models onto residuals that are already close to noise risks overfitting and wishful thinking about small, unstable gains in performance.

Beware of naive implementations: - applying complex models directly to raw, non-stationary data can give misleading results; - neglecting transformations and residual diagnostics can lead to overconfident conclusions.

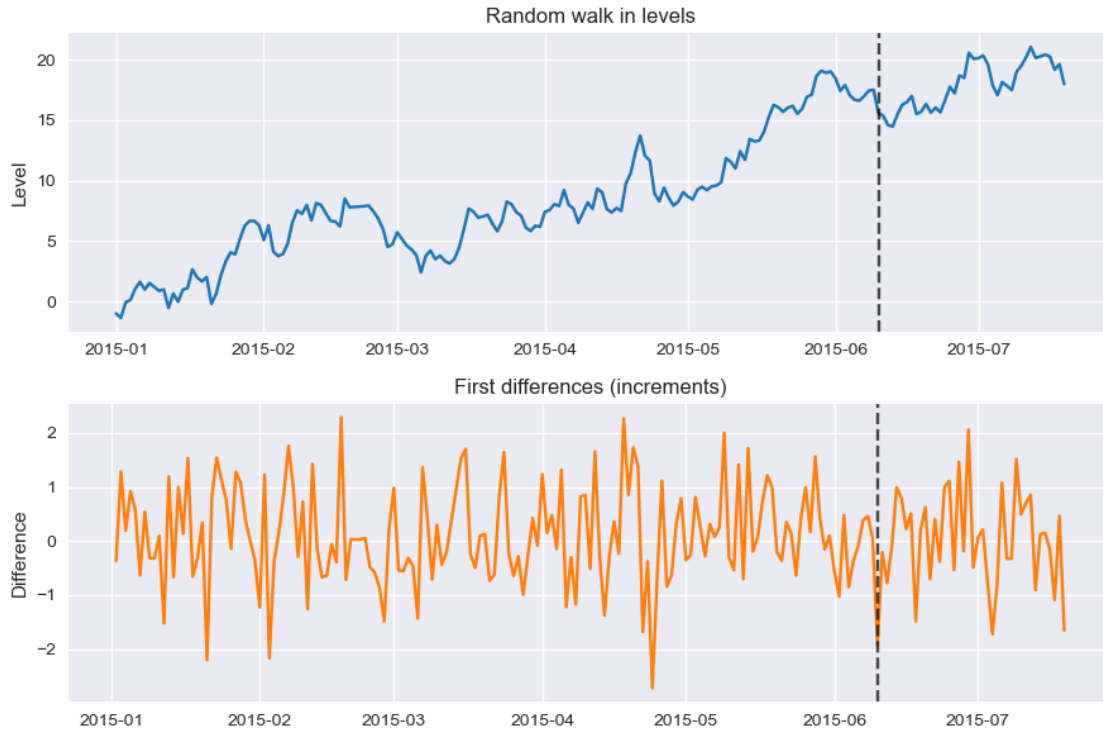
1.3.3 3.3 Levels vs differences and illusion of skill

If we only evaluate models on levels for highly autocorrelated series, we risk mistaking persistence for genuine forecasting skill. Comparing performance on levels versus differences helps reveal this:

- on levels:
 - persistence and simple AR models often achieve high R^2 and low error;
 - complex ML models may show apparent gains by capturing trend and persistence;
- on differences:
 - if differences are close to white noise, all models should perform similarly poorly;
 - any strong apparent skill may be suspicious or due to leakage.

To make this more concrete, we next look at a simple random-walk example where we plot the series in levels and in first differences. We also compute a basic error metric, the root mean squared error (RMSE), for one-step-ahead persistence on levels and for a baseline that predicts zero on first differences. In this idealised setting the two RMSE values turn out to be essentially identical, because both error sequences reduce to the same underlying shocks. Later, in section 7, we return to error metrics in more detail and use such comparisons between levels and differences to help distinguish apparent skill on highly autocorrelated levels from genuine ability to forecast innovations.

```
RMSE on levels for persistence:    0.913
RMSE on differences (predict 0): 0.913
```



1.4 4. Classical forecasting models

Classical linear models provide structured ways to capture dependence over time. We focus on: - autoregressive (AR) models; - moving-average (MA) models; - combined ARMA models; - ARIMA models that include differencing.

These models: - impose linear structure on lagged values and shocks; - are well-understood and widely implemented; - serve as baselines and building blocks for more complex approaches.

1.4.1 4.1 AR, MA, ARMA, ARIMA as linear models with memory

At a high level:

- AR(p) models:
 - regress y_t on past values $y_{t-1}, \dots, y_{t-p}$;
 - capture dependence through lagged observations.
- MA(q) models:
 - express y_t as a linear combination of current and past shocks $e_t, e_{t-1}, \dots, e_{t-q}$;
 - capture dependence through lagged innovations.
- ARMA(p, q) models:
 - combine AR and MA components:
 - allow dependence on both past values and past shocks.
- ARIMA(p, d, q) models:
 - apply d differences before fitting an ARMA model;
 - handle certain forms of non-stationarity in levels by modelling differenced series.

All of these can be seen as linear models with memory: - inputs are lagged values or shocks; - outputs are current or future values; - estimation typically uses likelihood or least squares under stationarity assumptions.

1.4.2 4.2 Assumptions and when they work well

Classical models work well when: - the series (or an appropriately transformed version) is approximately stationary; - dependence is primarily linear and captured by a modest number of lags; - variance is relatively stable over the periods of interest.

In such cases, ARIMA-type models: - provide interpretable structures; - can produce competitive forecasts; - support principled diagnostics and model comparison (for example residual analysis, information criteria).

1.4.3 4.3 Limitations and failure modes

Limitations arise when:

- dynamics are strongly non-linear or chaotic;
- heavy tails or extreme events play a central role;
- regime changes and concept drift are frequent;
- multivariate and high-dimensional dependencies are important.

In these settings, simple linear models may:

- understate uncertainty;
- miss important interactions;
- give a false impression of stability.

They remain useful as baselines and interpretable starting points, but more flexible models and robust validation are often required.

To illustrate how a simple AR model behaves when its assumptions are satisfied, we now fit an ARIMA(1,0,0) model to a synthetic AR(1) series where the true data-generating mechanism is known. The printed summary output shows estimated coefficients, information criteria, and a few standard diagnostics. We then inspect residual plots and autocorrelation to see whether the fitted model has captured the main linear structure, and compare its out-of-sample performance with a persistence baseline.

SARIMAX Results						
=====						
Dep. Variable:	AR(1) series		No. Observations:		160	
Model:	ARIMA(1, 0, 0)		Log Likelihood		-209.949	
Date:	Thu, 24 Sep 2026		AIC		425.898	
Time:	13:22:54		BIC		435.124	
Sample:	01-01-2012		HQIC		429.645	
	- 06-08-2012					
Covariance Type:	opg					
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	0.3757	0.206	1.828	0.068	-0.027	0.779

ar.L1	0.6552	0.060	10.870	0.000	0.537	0.773
sigma2	0.8049	0.087	9.248	0.000	0.634	0.975

```
=====
```

```
===
```

```
Ljung-Box (L1) (Q):                0.28   Jarque-Bera (JB):
```

```
0.14
```

```
Prob(Q):                0.60   Prob(JB):
```

```
0.93
```

```
Heteroskedasticity (H):            0.96   Skew:
```

```
-0.02
```

```
Prob(H) (two-sided):            0.88   Kurtosis:
```

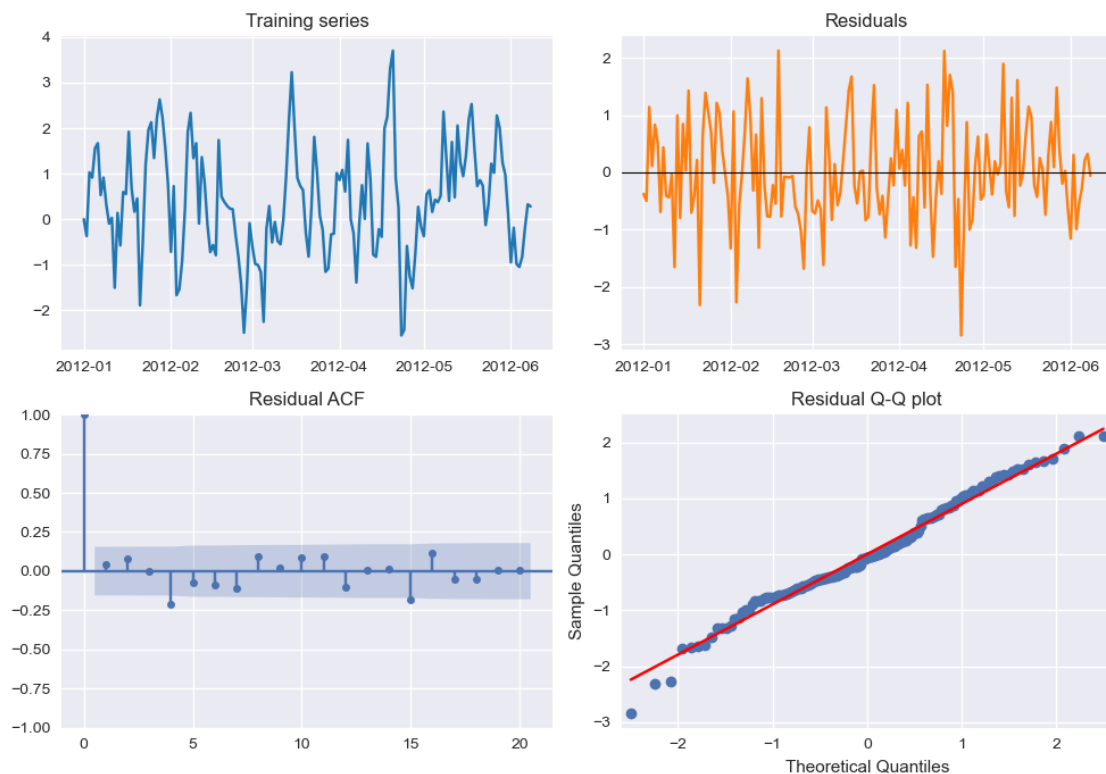
```
3.14
```

```
=====
```

```
===
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).



Test performance

ARIMA(1,0,0): MAE=0.971, RMSE=1.192

Persistence : MAE=0.799, RMSE=0.993

1.5 4.4 Spectral views of seasonality and structure

Autocorrelation and ARMA-type models describe dependence in the **time domain**. The **spectral density** describes how the variance of a stationary series is distributed across different **frequencies**.

At a high level:

- A (weakly) stationary series has an autocorrelation function that decays suitably with lag.
- Under these conditions, one can apply a Fourier transform to the autocovariance function to obtain the **spectral density**.
- The spectral density tells us how much of the series' variance is associated with slow oscillations (low frequencies) versus rapid oscillations (high frequencies).

Two important facts (under the usual regularity conditions):

- The spectral density is (informally) the 'Fourier transform' of the autocovariance function.
- The total variance of the series equals the integral of the spectral density over the relevant frequency range. This means we can interpret the spectrum as showing how different frequencies contribute to overall variability.

1.5.1 4.4.1 Periodograms as noisy spectral estimates

In practice we only observe a finite sample, so we cannot directly recover the true spectral density. Instead we compute a **periodogram**, which is a finite-sample estimate of the spectral density obtained by applying a discrete Fourier transform to the data.

Key points:

- For finite samples, periodograms are typically quite noisy (they can be 'spiky' or 'messy').
- Smoothing (for example averaging across neighbouring frequencies) is often used to obtain a more interpretable estimate of the underlying spectrum.
- Because of sampling variability, even a true white-noise series will not produce a perfectly flat periodogram unless the sample is extremely long.

For this module, you do **not** need to work with Fourier transforms at the mathematical or implementation level. We will rely entirely on standard library routines that:

- compute a periodogram from a univariate series;
- optionally apply simple smoothing to highlight dominant peaks.

1.5.2 4.4.2 What spectra reveal (and what they do not)

Spectral views support several practical tasks:

- **Seasonality and periodic structure:**
 - A series with strong seasonality (for example daily or weekly cycles) will show clear peaks at the corresponding frequencies.
 - A series close to white noise will have a relatively flat spectrum.
- **Multiple overlapping cycles:**
 - A time series formed by adding several sinusoidal components with different frequencies plus noise can look visually 'messy' in the time plot.
 - Its periodogram can reveal distinct peaks at the underlying frequencies, making the structure much clearer.

- **Variance decomposition:**

- Because the integral of the spectral density equals the variance, we can see how much variability is concentrated at particular frequencies (for example whether low-frequency trend-like components dominate, or whether high-frequency fluctuations are important).

For example, we can construct a simple synthetic series from three or four sine waves with added noise, plot it in the time domain, and then compute its periodogram. The time plot will often look visually ‘messy’, whereas the periodogram reveals distinct peaks close to the underlying component frequencies. A small example appears later in subsection 4.4.5.

1.5.3 4.4.3 White noise and flat spectra

A white-noise process (mean zero, constant variance, no autocorrelation) has a **flat** theoretical spectral density: all frequencies contribute equally to the variance. This is analogous to ‘white’ light, which contains all visible frequencies with equal intensity.

Important cautions:

- In finite samples, even a true white-noise process will not produce a perfectly flat periodogram.
- A relatively flat empirical spectrum is **suggestive** of white-noise-like behaviour, but not a formal proof.
- Certain processes, such as GARCH models, can have uncorrelated increments (and hence a flat spectrum) but display strong dependence in higher-order properties like volatility. Spectral methods based on autocovariance alone cannot detect this kind of structure.

1.5.4 4.4.4 Cross-spectra and other transforms (brief note)

For multivariate series, one can define **cross-spectral densities** based on cross-covariance functions. These are complex-valued functions that encode both the strength and phase relationship between two series at different frequencies. Interpreting cross-spectra is more involved and beyond the scope of this module, but the basic idea mirrors the univariate case: we ask at which frequencies two series co-move.

From a mathematical perspective, classical spectral analysis decomposes a time series into sums of sine and cosine waves of different frequencies (Fourier series). Other transforms are possible, such as wavelet transforms, which use localised waveforms that can adapt to changes in behaviour over time. Wavelet methods offer a richer time–frequency representation but are beyond the scope of this module.

In this module we will:

- use simple periodograms and smoothed spectral estimates to confirm obvious seasonality (for example daily/weekly cycles in energy demand or weather);
- contrast series with clear periodic structure against series that look random-walk-like;
- use these views to inform the choice of seasonal baselines (such as seasonal naive with period m) and the plausibility of seasonal ARIMA models.

1.5.5 4.4.5 Example: two complex series with similar time plots but different spectra

To make the spectral ideas more concrete, consider two synthetic series built from several sine waves plus noise:

- both series look relatively complex and noisy in the time domain, and their plots can be hard to distinguish by eye;
- but they are constructed from different sets of underlying frequencies.

In the time domain:

- it is not obvious which periodic components are present, or even whether the two series differ in any systematic way;
- for many practical problems, visual inspection of the time plot is not enough to separate different types of signals.

In the frequency domain, however, their periodograms reveal different patterns of peaks:

- each series shows peaks at its own set of dominant frequencies;
- the spectral differences are much easier to see than in the raw time-domain plots.

This illustrates that:

- two series can look similar in the time domain but have clearly different spectral signatures;
- spectral methods can be useful for classification problems where the key differences lie in frequency content rather than obvious time-domain shape.

A classic example is discriminating between different types of seismic events, such as natural earthquakes versus nuclear explosions. The time-domain waveforms may be hard to distinguish by eye, but their spectra can show systematic differences that are exploited in detection and classification pipelines.

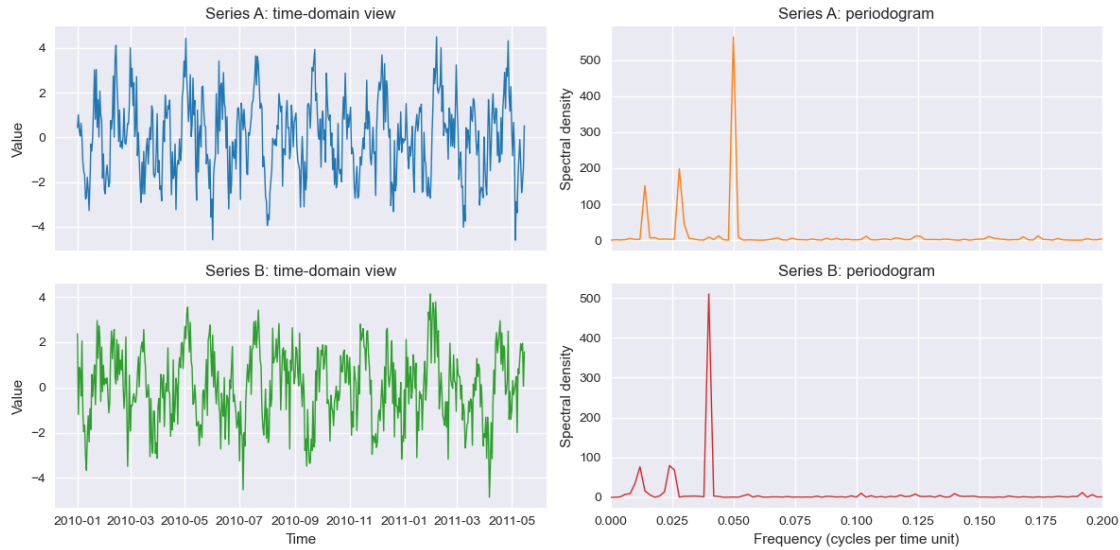
In modern applications, some machine learning models:

- work directly on frequency-domain or time-frequency features (for example periodograms, spectrograms, wavelet coefficients);
- or combine time-domain and spectral features as inputs;
- or operate on residual or de-trended series rather than raw levels, to focus on stationary components where spectral analysis is more meaningful.

This is conceptually similar to our earlier emphasis on working with differences or residuals when series show random-walk-like behaviour. Often we first remove obvious trend and seasonality, and then analyse the remaining component in the frequency domain.

In the following code cell, we:

- construct two series (Series A and Series B), each as a sum of several sines with different periods plus noise;
- compute their periodograms;
- plot both time-domain series and both spectra in a single figure with four subplots:
 - the two time series on the left;
 - the two corresponding spectra on the right.



1.5.6 4.4.6 Spectral density, autocorrelation, and non-linear dependence

It is helpful to remember that, for stationary series, the spectral density is derived from the autocovariance (or autocorrelation) function via a Fourier transform. In other words:

- the spectrum reflects how second-order dependence (linear correlations at different lags) is distributed across frequencies;
- it does not directly capture non-linear forms of dependence.

This connects to the earlier discussion of white noise and GARCH-type processes:

- a series with uncorrelated increments and constant variance has a flat theoretical spectral density, and is often called white noise;
- certain econometric models such as GARCH can produce series with uncorrelated increments (and hence a flat spectrum) but strong dependence in higher-order properties, such as volatility clustering.

In such cases:

- spectral methods based purely on autocorrelation cannot detect the non-linear dependence;
- one must use tools that are sensitive to conditional variance and other higher-order behaviour.

This illustrates that:

- spectral analysis is powerful for revealing linear correlation structure and periodic components;
- but econometric time series may exhibit non-linear dynamics that require additional modelling tools beyond simple spectral and ARMA-type methods.

1.5.7 4.5 Choosing ARIMA orders in practice

In the illustrative code above, we fit an ARIMA(1,0,0) model to a simple AR(1) series where we know the true data-generating mechanism. In real applications the correct orders (p, d, q) are not

known and must be chosen from the data.

Common strategies include:

- **Grid search over (p, d, q) :**
 - specify reasonable ranges, for example $p, q \in \{0, 1, \dots, P_{\max}\}$ and $d \in \{0, 1, 2\}$;
 - fit models across this grid;
 - use information criteria such as AIC or BIC to select a model that balances fit and complexity.
- **Automated procedures:**
 - functions such as `auto_arima` in some libraries implement heuristics that combine stationarity tests, differencing choices, and information criteria to propose an ARIMA order.
- **Diagnostic checks:**
 - examine residual autocorrelation and other diagnostics to see whether the chosen orders adequately capture the dependence structure.

In this module we will not go deeply into order-selection theory, but it is important to recognise that:

- ARIMA orders are typically **estimated or selected**, not known in advance;
- different choices can lead to different forecasts, especially in small samples;
- it is wise to compare ARIMA forecasts against simple baselines (mean, persistence, seasonal naive, simple AR) under time-aware validation.

1.6 5. Time series as supervised learning

Many time series forecasting methods can be viewed as supervised learning problems. This perspective allows us to: - use general ML tools (trees, ensembles, neural networks) on lagged feature vectors; - keep temporal structure explicit through feature design and validation.

1.6.1 5.1 Lagged-feature formulation

A supervised learning problem has: - inputs (features); - targets (labels); - a model that learns a mapping from inputs to targets based on training data.

For time series, we typically: - take past observations as inputs; - predict future values as targets.

Examples:

- lag approach:
 - use $(X_{t-1}, X_{t-2}, \dots, X_{t-L})$ as input,
 - predict X_t or X_{t+h} as target;
- sliding window variations:
 - define windows of length L that slide along the series;
 - treat each window as an input vector and the next point (or next few points) as target.

This connects classical time series models (such as AR models) and general supervised learners: - AR models can be seen as linear regressions on lagged values; - tree-based or neural-network models can operate on the same lagged feature vectors to capture non-linear relationships.

Window length L and forecast horizon h : - control how much history is used; - affect the difficulty of the prediction task; - must reflect domain knowledge (for example relevant memory length, seasonal period).

Conceptually, most supervised learning models used here do one simple thing:

- they receive a vector of past values and possibly exogenous variables as input;
- they learn a function that maps these inputs to a future value;
- learning consists of adjusting internal parameters to minimise a loss (for example squared error) on the training window.

Different models (linear regression, trees, Random Forests, neural networks) differ mainly in the class of functions they can represent and in how they fit parameters. The forecasting logic - 'use past to predict future under a chosen loss and validation scheme' - is the same.

1.6.2 5.2 Feature types for forecasting

Once we treat time series forecasting as supervised learning, feature engineering becomes central. Typical features include:

- lagged values: $X_{t-1}, X_{t-2}, \dots, X_{t-L}$;
- rolling statistics: moving averages, rolling variance, rolling minima or maxima;
- calendar features: day of week, month, quarter, holidays, working days vs weekends;
- exogenous variables: weather, prices, control signals, other related series.

When constructing features we must: - ensure that no future information leaks into inputs; - align exogenous variables correctly in time; - consider whether to work on levels or differences, depending on stationarity and random-walk behaviour.

Leakage is a central risk: - using future target values or future exogenous variables as inputs; - mixing training and evaluation periods when computing rolling features.

Careful feature design and temporal splitting are needed to avoid these pitfalls.

1.6.3 5.3 ML models for forecasting

In this module we mostly use tree-based ensembles (Random Forests, gradient boosting) as examples of flexible models. Intuitively:

- a **decision tree** learns simple if-else rules on lagged values (for example 'if X_{t-1} is large and X_{t-2} is small, predict a high value');
- a **Random Forest** averages many such trees, each built on slightly different subsets of the data and features, to reduce variance and capture a range of patterns;
- a **gradient boosting model** builds trees sequentially, each new tree trying to correct the remaining errors of the previous ensemble.

All of them still follow the same basic principle: they adjust their internal rules to minimise prediction error on the training window, and we judge whether the resulting function generalises by using time-aware validation.

Once lagged-feature representations are defined, many supervised learners can be applied: - tree-based models: decision trees, Random Forests, gradient boosting methods; - linear and regularised models; - multilayer perceptrons (MLPs) on lag windows; - other architectures (for example recurrent or convolutional networks) in more advanced work.

These models can: - capture non-linear relationships between lagged features and future values; - accommodate interactions between lags and exogenous variables; - adapt to complex patterns that

classical linear models may miss.

However, their flexibility increases the risk of overfitting, especially with limited data. Strong baselines and careful validation are essential.

1.6.4 5.4 Flexible tools, not magic

Machine learning models do not automatically create predictability. They are powerful tools that must be: - embedded in sensible feature engineering and transformations; - evaluated properly with time-aware validation and baseline comparisons; - interpreted with attention to data quality and concept drift.

Several accessible video series provide intuitive explanations of machine learning and neural networks:

- CGP Grey’s ‘How Machines Learn’ and ‘How Machines Really Learn’:
 - explain how models learn patterns from data;
 - provide intuition about training, loss functions, and generalisation;
- 3Blue1Brown’s neural network playlist:
 - visual explanations of how neural networks represent non-linear functions;
 - introductions to backpropagation and gradient descent.

These resources can help students build intuition about how supervised learning algorithms operate, which is useful when thinking about: - feature engineering for time series; - model capacity vs overfitting; - why flexible models are not magic, but tools that must be evaluated carefully.

1.7 6. Temporal validation and leakage

Reminder (TS1 Section 4.4): train/validation/test and gaps TS1 introduced train/validation/test splits, leakage, and the idea of quarantine gaps:

Train: $[t_0, \dots, t_1]$, Gap: $[t_1 + 1, \dots, t_1 + g]$, Validation/Test: $[t_1 + g + 1, \dots, t_2]$.

In TS2 we do not repeat the full conceptual discussion. Instead, we show **how to implement** time-aware splits and walk-forward validation in code (see PCS956-TS-companion_B), and how to compare baselines and models under such splits.

Evaluation for time series forecasting must respect time. Random shuffling and standard k -fold cross-validation are usually inappropriate and can lead to severe leakage.

The use of machine learning for time series forecasting comes with specific pitfalls. Important lessons highlighted in practical discussions include:

- the danger of random train/test splits:
 - random k -fold cross-validation mixes past and future;
 - this allows future information to leak into training;
 - performance estimates become unrealistically optimistic;
- the need for time-aware validation:
 - training on past, validating/testing on future;
 - rolling-origin or expanding-window evaluation schemes;

- clear separation between training period and evaluation period.

Practical tutorials and blog posts emphasise strategies to avoid these pitfalls: - use walk-forward validation; - make sure the forecasting horizon and validation design match the intended use case; - compare machine learning models against simple baselines such as persistence and seasonal naive forecasts.

These sources often use examples based on: - random walks and unit-root behaviour; - Granger-style causality tests for multivariate series.

Together they reinforce the central message: - transformations of the time series under investigation might be required; - machine learning methods are powerful, but must be embedded in a careful forecasting and validation workflow.

1.7.1 6.1 Time-aware train/validation/test splits

Time-aware splits typically follow: - train on an initial block of past data; - validate on a subsequent period; - test on a final, held-out period.

Variants include: - expanding-window schemes: - progressively extend the training window as time advances; - evaluate forecasts at multiple cut points; - rolling-window schemes: - keep a fixed-length training window that moves through time; - suitable when very old data may be less relevant due to drift.

The choice of scheme should match: - the intended deployment scenario; - the forecast horizon; - the degree of concept drift expected.

Quarantine gaps: how big should the gap be? In TS1 we introduced the idea of a **quarantine gap** between training and validation or test periods:

$$\text{Train: } [t_0, \dots, t_1], \quad \text{Gap: } [t_1 + 1, \dots, t_1 + g], \quad \text{Validation/Test: } [t_1 + g + 1, \dots, t_2].$$

The gap length g is not fixed by theory; it is a **design choice** that depends on:

- how far temporal dependence extends (for example how slowly autocorrelations decay);
- how features are constructed (for example whether rolling statistics blur information across time);
- how strongly we want to reduce potential leakage from training into evaluation.

As a practitioner, one typically:

- inspects the autocorrelation and partial autocorrelation functions to see at what lags dependence becomes negligible;
- considers the longest look-back window used in feature engineering (for example a 30-day rolling mean suggests that a gap shorter than 30 days may still carry some leakage);
- chooses a gap long enough that:
 - evaluation points are not trivially influenced by the last part of the training window via slow dependence, and
 - rolling features or other transformations do not inadvertently use future data.

There is a trade-off:

- larger gaps reduce potential leakage but leave fewer observations for training and testing;
- very small or zero gaps may overstate performance if the evaluation period is still strongly tied to the training period through serial dependence or feature construction.

Exploring gap choices with simulated data A useful exercise is to explore different gap sizes on simulated data:

- generate a synthetic series with known dependence structure (for example AR(1), ARMA, or a more persistent AR process);
- fix a simple forecasting model (for example persistence, AR(1), or a small tree-based model on lags);
- evaluate out-of-sample error for different gap lengths g (for example $g = 0, 5, 10, 30$).

By comparing performance metrics across g , you can see:

- how much apparent performance drops when we increase the gap and reduce potential leakage;
- how sensitive your conclusions are to a particular choice of gap length.

This kind of simulation helps build intuition about why quarantine gaps are used and how they interact with serial dependence and feature engineering.

1.7.2 6.2 Why random k-fold splits are misleading

Random k -fold splits assume i.i.d. data and mix past and future: - training folds can contain observations that occur after those in validation folds; - model estimates implicitly use future information when predicting the past.

Consequences: - leakage of future information into training; - overoptimistic performance estimates; - models that appear strong in cross-validation but fail in real deployment.

For time series forecasting, random k -fold cross-validation should generally be avoided in favour of time-aware schemes.

1.7.3 6.3 Comparing naive and proper validation

One useful exercise is to contrast: - performance under naive random splits; - performance under proper temporal splits.

Typically: - random splits yield higher apparent accuracy and R^2 ; - temporal splits reveal more modest performance and highlight the difficulty of forecasting.

In small samples or particular parameter configurations, a single simulation may not follow this pattern exactly; this is due to finite-sample randomness rather than a failure of the underlying argument about leakage.

Such comparisons: - illustrate how much optimism random splits can introduce; - reinforce the need for honest evaluation in both research and deployment; - show that beating baselines under temporal validation is far more meaningful than achieving impressive metrics under random splitting.

Random split RF R^2 (leaky): 0.775
 Temporal split RF R^2 (proper): 0.551

1.8 7. Evaluation on levels vs differences

Evaluation for highly autocorrelated series must distinguish between: - forecasting levels y_t ; - forecasting differences or residuals (innovations).

Combining random-walk intuition (section 3) with baselines and temporal validation, we emphasise: - persistence can produce high apparent skill on levels; - genuine predictive structure should be visible in differences or residuals where appropriate; - metrics, especially R^2 , must be interpreted carefully.

1.8.1 7.1 Standard metrics for forecasting

Common point-forecast metrics include:

- mean absolute error (MAE):
 - average absolute deviation between forecasts and true values;
- root mean square error (RMSE):
 - square root of mean squared error, penalising larger errors more strongly;
- R^2 :
 - proportion of variance explained relative to a mean baseline.

Other metrics you will encounter in practice include:

- mean absolute percentage error (MAPE) and related percentage-based errors:
 - useful when scale-free comparisons are needed, but problematic when values are near zero;
- scale-dependent or relative errors (for example metrics normalised by the mean or by a naive baseline):
 - helpful when comparing across series with very different scales;
- quantile-based losses (for example pinball loss) for probabilistic forecasts:
 - used when we care about full predictive distributions or specific quantiles rather than single point estimates.

In this module we will mostly work with MAE, RMSE, and R^2 . MAE and RMSE are straightforward error measures; lower values indicate better performance. R^2 requires more care in forecasting settings, particularly for autocorrelated series and out-of-sample evaluation.

1.8.2 7.2 Interpreting R^2 and problems on levels

In many introductory courses, R^2 is met only in the setting of simple linear regression (one predictor, evaluated on the same data used for fitting). It is then often described as ‘the square of the correlation’ and assumed to lie between 0 and 1.

For general forecasting or regression models, evaluated out-of-sample, the definition is:

$$R^2 = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2},$$

where: - y_i are the true values in the evaluation set; - $\hat{y}_i$ are the model predictions; - $\bar{y}$ is the mean of the y_i in that evaluation set.

Interpretation: - $SS_{\text{tot}} = \sum (y_i - \bar{y})^2$ is the squared error of a mean baseline that always predicts $\bar{y}$;
- $SS_{\text{res}} = \sum (y_i - \hat{y}_i)^2$ is the squared error of our forecasting model.

So: - $R^2 = 1$ means perfect predictions; - $R^2 = 0$ means the model is no better than always predicting the mean; - $R^2 < 0$ means the model is worse than just predicting the mean on that target.

Two important consequences for time series:

1. R^2 can be negative. This is normal once we move beyond simple in-sample linear regression. It simply signals that, on the evaluation set, our model does worse than the trivial ‘predict the mean’ baseline.
2. High R^2 on levels does not imply real forecasting skill. For highly autocorrelated series (for example random-walk-like data), predicting the level y_t can look excellent because ‘tomorrow is similar to today’. A model that essentially copies yesterday’s value can achieve high R^2 on levels, even though it has little ability to forecast innovations or changes.

This connects back to section 3.3: if we only look at performance on levels for a highly autocorrelated series, we risk mistaking persistence for genuine predictive skill.

1.8.3 7.3 Strategies for robust evaluation

Robust evaluation strategies include:

- compare performance on levels vs differences:
 - strong gains on levels but no improvement on differences may indicate reliance on persistence;
- examine residuals and their autocorrelation:
 - residuals with strong remaining autocorrelation suggest underfitting;
 - residuals that look like white noise suggest limited remaining structure;
- benchmark against simple baselines:
 - mean, persistence, seasonal naive, simple AR.

Good performance should be visible: - relative to these baselines; - under temporal validation; - in both levels and differences where those targets are meaningful.

1.8.4 7.4 Case study: forecasting and cherry-picking

In practice, it is easy to cherry-pick favourable examples: - a tree-based or other ML model can look very strong on a single series; - performance can change dramatically across different series, time periods, or targets.

Case studies should highlight: - how evaluating only on one favourable dataset encourages overconfident conclusions; - how performance can drop when: - moving to different series; - using proper temporal splits; - switching from levels to differences or residuals; - how using multiple series and honest temporal splits tests robustness and helps avoid overstating skill.

1.9 8. Mini-project support: forecasting and validation

The time-series mini-project associated with this module requires students to work with at least one temporal dataset and to design, implement, and evaluate at least one modelling approach on that dataset. The data can come from the student’s own research (if suitable for sharing), from

openly available sources, or from the curated teaching/example datasets (including synthetic or anonymised variants) provided in the course materials. The main task may involve forecasting, classification (for example event type), anomaly or change-point detection, or a simple structural modelling question, depending on the student's interests.

1.10 8.1 Choosing baselines and main models

Students should: - select at least one baseline: - persistence or seasonal naive from section 2; - optionally a simple AR or mean baseline; - select one or more main models: - classical (for example ARIMA); - or ML-based (for example tree-based or neural models on lagged features).

Choices should be justified: - why the baseline is appropriate for the series; - why the main model is plausible given stationarity, trend, and seasonality; - how complexity is balanced against data size and quality.

1.10.1 8.2 Planning temporal splits and target choice

Students must design train/validation/test splits that: - respect time order (section 6); - reflect the intended forecasting horizon and deployment scenario.

They should also decide whether to model: - levels; - differences; - residuals after decomposition.

Justification should draw on: - stationarity and random-walk behaviour (section 3); - the evaluation discussion in section 7; - data-quality considerations from Lecture TS1.

1.10.2 8.3 Documenting successes and failures

Good scientific practice includes reporting both successful and unsuccessful models: - models that fail to beat baselines under temporal validation; - series that appear close to random-walk-like, where baselines are hard to beat; - experiments that reveal strong concept drift or regime changes.

Showing that a series is close to random-walk-like, with limited exploitable structure, is a valuable finding. Projects should emphasise clear narratives and honest conclusions rather than only 'nice' results.